

Lab Answer Key — RAG Beyond the Demo

Task 1 — Explore the RAG Pipeline

Q1 — What determines section/clause values?

HEADING_RE = re.compile(r"^\s*(\d+)\.\s+([A-Z][^\.]{2,60})\$") inside NumberedHeadingChunker. The critical design detail: the title portion must start with a capital letter, contain no period, and stay under 60 characters. This was **not** the original regex — an earlier version (`^\s*(\d+)\.\s+(.+)$`) matched any digit-dot-text line, which is what caused the false-heading bug on Slide 7. **Common wrong answer:** "it matches any line starting with a number and a period" — this was true of the buggy first version, not the corrected one. The distinction (title-shaped vs. sentence-shaped) is the actual lesson.

Q2 — Why does DocxExtractor convert to PDF first? DOCX is a flow-based document format with no fixed page boundaries in its raw XML — pages only exist once something renders/paginates the content. PdfExtractor already had correct, real per-page extraction via pypdf, so DocxExtractor reuses it by converting via LibreOffice headless first, rather than inventing a second, less accurate way to estimate pages. **Why correct:** this satisfies FR-1's "≥2 input formats" requirement without duplicating page-detection logic — both formats are independently supported adapters implementing the same DocumentExtractor port; the internal conversion is an implementation detail.

Q3 — Idempotency field. Document.content_hash — SHA-256 of the raw file bytes, computed in IngestDocumentUseCase.execute() before extraction runs. If the stored hash matches, the use case returns IngestResult(status="unchanged") without re-extracting, re-chunking, or re-embedding. **Why this matters pedagogically:** idempotency here is content-based, not filename-based — renaming a file with unchanged content would still re-ingest it (a reasonable, documented tradeoff, not a flaw).

Task 2 — Run the RAG Pipeline

Expected: every previously-unseen corpus file reports ingested (N chunks); re-running the same command a second time should show unchanged, skipped for every file — this is worth having trainees actually verify by running it twice.

Common wrong answer: assuming a nonzero chunk count on the second run means duplication is happening. The correct check is that the *total* row count in `persistence_documentchunk` does not increase between runs — this was the actual idempotency bug found and fixed during this project's own development (a resumed pipeline step was still calling `record_agent_run`, producing duplicate rows — analogous but not identical to this ingestion-level check).

Task 3 — Inspect Retrieval

Q1 — Document/section plausibility. Trainees should find that a query like "What is the deductible for the auto comprehensive policy?" retrieves a document whose filename contains `auto_comp` and a section resembling "6. Limits of Liability". If it instead retrieves an unrelated policy type, this reproduces the boilerplate-collision bug from Slide 12 — worth flagging explicitly if observed, since it would mean the contextual- embedding fix regressed.

Q2 — Adversarial case score. Out-of-corpus cases (e.g. "capital of France") should show a `dense_sim` value in the mid-range (roughly 0.4–0.7 was observed during development) — the correct teaching point is that this is **not** near zero, which surprises most trainees expecting "unrelated text = low similarity." This is the anisotropy limitation from Slide 15.

Q3 — top_k. `RetrieveChunksUseCase.execute()` defaults `top_k=5` for the final result set, but pulls `top_k=20` candidates internally from each retriever before fusion. Raising the final `top_k` trades precision (more marginal/irrelevant chunks reach the LLM's context) for recall (less chance of missing a relevant chunk) — and increases both LLM input token cost and latency. **Common wrong answer:** "raising `top_k` always improves answer quality" — this is exactly Misconception #2 in the common-mistakes document; more context can dilute the LLM's attention on the actually-relevant chunk.

Task 4 — Ask a Grounded Question

Trainees should observe a citation object with all of `chunk_id`, `document`, `policy_version`, `section`, `clause`, `page`, `excerpt` populated (see `format_citation()`). If any field is empty, that's worth investigating as a real defect, not expected behavior — every field is meant to be populated for every returned chunk.

Why correct: citations exist specifically so an adjuster can verify the source without trusting the LLM's paraphrase — the structured fields, not just an inline footnote-style reference, are what make the citation independently checkable.

Task 5 — Analyze a Failure Case

Expected reasoning: "What is the deductible?" with no policy named is genuinely ambiguous — the information exists in the corpus (every policy has a deductible), the ambiguity is about *which* policy the user means, not about missing evidence.

Correct diagnosis: this is **not** a pure retrieval problem — retrieval correctly finds a deductible; the gap is a missing disambiguation step that would need to ask "which policy?" before answering, which belongs at the agent/conversation layer, not the retrieval-threshold layer. A trainee who says "just raise the refusal threshold higher" should be pushed to explain why that would also start refusing legitimate, answerable questions — because raw similarity alone cannot distinguish "ambiguous but answerable" from "genuinely out of corpus."

Common wrong answer: treating this the same as the out-of-corpus case (Slide 15) and proposing the same fix. The two failure modes look similar in symptom (both might not refuse) but have different correct fixes.

Stretch Challenge Answers

Stretch 1 — RRF Parameter Change

With $\text{rrf_k}=10$, a rank-1-in-both-lists chunk scores $1/11 + 1/11 \approx 0.182$. With $\text{rrf_k}=60$, the same chunk scores $1/61 + 1/61 \approx 0.033$ — roughly 5.5x smaller. This directly reproduces the original bug documented in ADR-005: a fixed refusal threshold that was reasonable at one rrf_k becomes either unreachable or trivially exceeded at another, because the *scale* of the fused score is rrf_k -dependent, not an absolute property of match quality. This is precisely why refusal was ultimately moved off the fused score entirely.

Stretch 2 — New Adversarial Case

There's no single correct query, but a strong answer identifies a genuinely new *category* not already covered (e.g., a query naming a real policy type that doesn't exist in the corpus, or a query mixing two real policy types in a way that could retrieve either). Weak answers just paraphrase an existing golden-set case. Full credit requires the trainee to actually run it and report the real observed behavior, not a predicted one.

Stretch 3 — Indirect Prompt Injection

The correct expected result is that the injected instruction is **not** followed — the system prompt for each agent explicitly frames retrieved text as evidence to summarize, not as instructions to obey, and nothing in the pipeline ever concatenates retrieved text into a position where the LLM would interpret it as a system/developer message. If a trainee's test *does* show the instruction being followed, walk back through exactly how their test document's text was phrased and where it landed in the prompt — this is a genuine, reportable finding, not necessarily a lab mistake, and mirrors exactly the kind of case the brief requires in the security evaluation set (≥ 3 injection cases the system demonstrably resists, including at least one indirect case).